LMCache

Introduction to LMCache

Why LLMs need a KV cache, and why the cache needs its own layer

Karsten Wade · LMCache SF Bay Meetup · Samsung, San Jose · 22 October 2026

IF YOU REMEMBER THREE THINGS

The short version of this talk

1

LLMs read the same text again and again. The KV cache is how an engine avoids that work.

2

GPU memory cannot hold it. The KV cache is now state that must be stored, moved, and shared.

3

LMCache is the open, engine-neutral layer that does this. It started as research and grew through its users.

PRIMER · HOW A MODEL ANSWERS

Before a model writes one word, it reads the whole prompt

Prefill · read every input token, all at once

`You are a helpful assistant.` `[ 38-page policy document ]` `How many leave days do I get?`

Decode · write one token at a time

`You` `get` `20` `days.`

On a long prompt, prefill is most of the compute, and it sets how long you wait for the first word.

While it reads, the engine keeps notes: the KV cache

For every token, in every layer, attention computes a **key** and a **value**. The engine saves them.

Next token, it reads the saved notes instead of the whole prompt. This is why decode is fast.

Keep the notes after the request ends, and the next request can skip prefill too.

HOW BIG ARE THE NOTES?

128 KiB

per token, Llama 3.1 8B

```
2 (K,V) × 32 layers × 8 KV heads ×
128 dims × 2 bytes
```

A 100,000-token context ≈ 12 GiB

And we send it the same text again and again

every request	the same system prompt	+ new
every turn	the whole chat so far	+ new
every question	the same document (RAG)	+ new
every agent step	tools, plan, and history	+ new

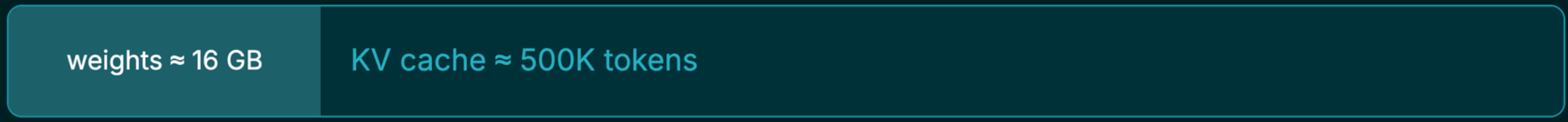
The text did not change. The engine reads it again, and you pay again.

> “KV-cache hit rate is the single most important metric”
>
> Yichao “Peak” Ji, Manus, on running production AI agents, July 2025

WHY IT MATTERS · THE LIMIT

The notes live in the most crowded memory on the machine

One 80 GB GPU serving Llama 3.1 8B



What is left after the weights: about 500,000 tokens of notes, shared by every active user

When it fills

The engine evicts the oldest notes to make room.

Next time

That text is read again, at full prefill cost.

Meanwhile

CPU memory and disks in the same box sit mostly idle.

Weights: 8B parameters × 2 bytes. Tokens: (80 − 16) GB ÷ 128 KiB, before activations and overhead.

WHY IT MATTERS · THE SHIFT

The KV cache has become state to store and move

It was designed for

One request

One engine

One GPU’s memory

Deleted when the answer ends

It is now used for

Prefix reuse across requests and users

Offload to CPU memory, disk, and remote storage

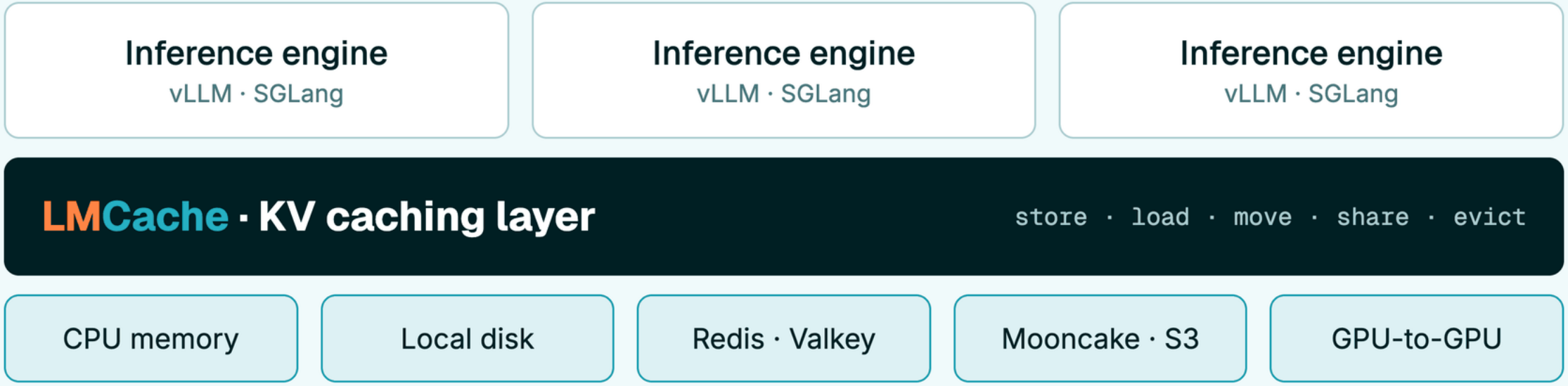
Transfer between engines: prefill on one GPU, decode on another

Hand-offs between agents that share context

State needs a data layer. Engines did not have one.

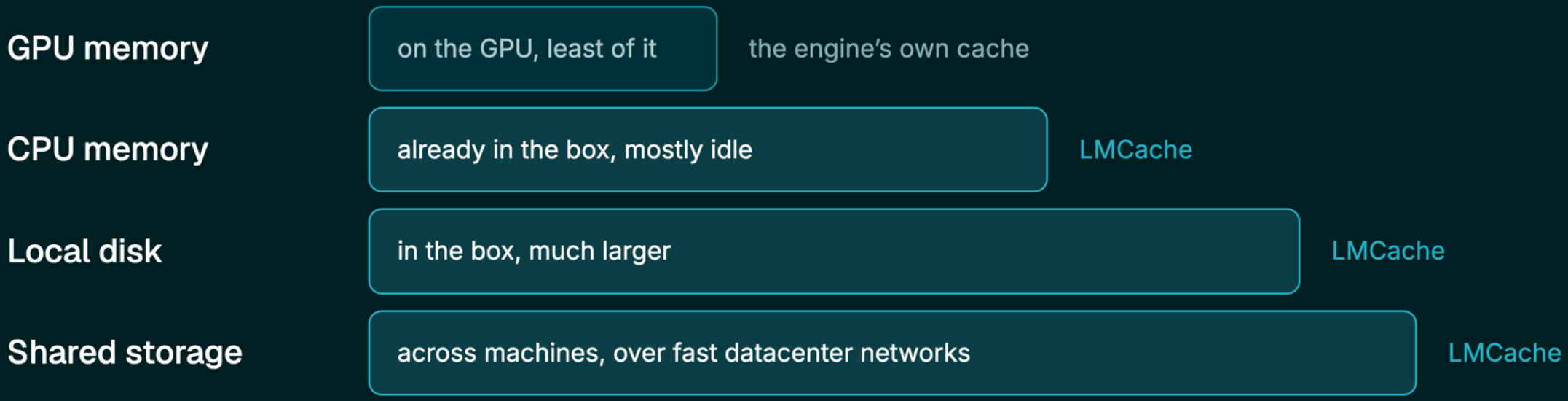
WHY IT MATTERS · THE MISSING PIECE

What was missing: a layer between engines and storage



WHAT LMCACHE ADDS · KEEP

LMCache keeps the notes instead of deleting them



Notes that leave the GPU move down a tier and stay ready. Loading them back costs far less than prefill and happens at comparable speeds.

WHAT LMCACHE ADDS · SHARE

Read once, reuse on one engine, many engines, or a fleet

one engine

Offload and reuse

The same prefix, across requests and users, survives GPU eviction.

many engines, one machine

One shared cache

Engines share a cache process next door, so it survives an engine crash.

across machines

Transfer and split

Move notes peer to peer, or prefill on one GPU and decode on another.

WHAT LMCACHE ADDS · RESULTS

Loading notes beats reading again

15×

Up to 15× higher throughput, vLLM with LMCache

Multi-round QA and document analysis

1.9–8.1×

Lower time to first token

Against vLLM's own CPU offload and commercial alternatives

1.5–1.8×

Lower mean time to first token with prefill and decode split

Against vLLM's native disaggregation

Source: Liu et al., "LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference," arXiv 2510.09665. Results depend on workload and reuse rate.

WHAT LMCACHE ADDS · OPEN AND NEUTRAL

Change engines or storage without losing your cache

ENGINES	PLATFORMS	STORAGE	HARDWARE
vLLM	NVIDIA Dynamo	CPU · local SSD	NVIDIA
SGLang	llm-d	Redis · Valkey	AMD
	KServe	Mooncake · S3	Ascend
	vLLM Production Stack	NIXL · GDS	TPU

LMCache plugs into vLLM through the KV connector API, a stable interface. When the engine changes its scheduler or memory, LMCache needs few changes.

WHERE IT CAME FROM · THE PROBLEM

It began with one question: what if KV cache moved like any other data?

2023 · SIGCOMM 2024

CacheGen

Compress and stream KV cache over the network

May 2024

LMCache

One framework to hold the lab's ideas, then opened to the ecosystem

2024 · EuroSys 2025

CacheBlend

Reuse cached chunks that are not a prefix, as in RAG

Junchen Jiang's lab at the University of Chicago. Then industry asked for speed, stability, and engine support.

WHERE IT CAME FROM · THE COMMUNITY

Users became contributors, and the project grew with them

Sep 2024

First public results with vLLM

Aug 2025

5,000 GitHub stars

Oct 2025

Joins the PyTorch Ecosystem

Oct 2026

More than 12,100 stars and 2,100 forks

Community contributors added eight storage backends and support for NVIDIA, AMD, Ascend, and TPU. The layer is neutral because many vendors built it.

Sources: blog.lmcache.ai; pytorch.org/blog; github.com/LMCache/LMCache (October 2026); LMCache technical report.

NEXT · 2:45 PM

After the break: hands-on with Samuel Shen and mini-lmcache

A public test server, live API endpoints to benchmark, and an environment that stays open after today. To try LMCache on your own machine:

```
$ pip install lmcache vllm
$ vllm serve Qwen/Qwen3-4B-Instruct-2507 \
--kv-transfer-config '{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_both"}'
```

Questions

PROJECT	github.com/LMCache/LMCache
DOCS	docs.lmcache.ai
COMMUNITY	lmcache.ai → Slack
PAPER	arxiv.org/abs/2510.09665

LMCache